Software Engineering Intelligence Platform

Security Track — Application & AI Security

Professional Threat Modeling Package

Critical Assets • Attack Surface • Initial Threat Model • Trust Boundaries • Threat Register • Security Requirements

Document purpose

This document translates the project architecture and the Security Track responsibilities in the supplied project specification into an initial security threat-modeling package. It deliberately uses the project's stated scope and terminology. Where the project document does not specify a concrete technology or protocol, this document marks the item as an assumption/verification point rather than presenting it as an implemented fact.

Source scope

The supplied project document defines the MVP as a software-company-focused SaaS using Meeting Notes, Requirements/Documents, and GitHub as initial data sources; Jira/Azure DevOps and Slack are roadmap items. The architecture is Flutter → ASP.NET Backend → AI Services, with a cross-cutting Security Layer including DLP, Prompt Injection Defense, AuthN/AuthZ, Monitoring and Vulnerability Management. □filecite□turn1file0□L48-L53□ □filecite□turn1file0□L111-L135□

1. System Scope & Security Boundary

In scope for the initial threat model: Flutter client, ASP.NET backend, event-driven processing, multi-tenant data layer, AI pipeline (NLP, Knowledge Graph, embeddings/vector database, RAG, LLM, ML models), security middleware, external company data sources, authentication/authorization, real-time notifications, APIs/webhooks, secrets and dependency supply chain.
MVP data sources: GitHub, Meeting Notes, Requirements/Documents. Jira/Azure DevOps and Slack are explicitly roadmap extensions, so they are not treated as deployed MVP components. □filecite□turn1file0□L48-L53□

Architecture basis

Layer / Component	Documented role	Security relevance
Flutter App	Dashboard, projects, tasks, AI Chat, graph visualization, notifications; SignalR real-time; secure local storage; offline mode + sync	Client compromise, token/local-data exposure, authorization bypass, reverse engineering
ASP.NET Backend	Clean Architecture + CQRS, Event Bus, SignalR, Multi-Tenant Data Layer, AuthN/AuthZ/Roles/Organizations, APIs	Primary trust boundary; tenant isolation; API access control; webhook abuse
Event Bus	RabbitMQ/Kafka; async background processing	Event injection, replay/duplication, queue abuse, poisoned downstream data
AI Services	NLP, Knowledge Graph, embeddings/vector DB, RAG, LLM, ML prediction	Prompt injection, unauthorized retrieval, data leakage, poisoning, model abuse
Security Layer	DLP, Prompt Injection Defense, AuthN/AuthZ, Monitoring & Vulnerability Management	Cross-cutting preventive/detective controls
External Sources	GitHub; Meeting Notes; Requirements/Documents	Untrusted/partially trusted content can enter processing and retrieval pipelines

2. Critical Assets Inventory

The project document explicitly identifies critical assets including GitHub data, employee information, AI conversations and secrets. filecite turn1file0 L244-L248 The following expands those categories into security-reviewable assets without asserting fields that the project specification has not defined.

ID	Critical Asset	Examples / Scope	Security properties
A-01	Company / Tenant Data	All data belonging to one software company	Confidentiality, integrity, tenant isolation
A-02	GitHub Data	Commits, PRs, branches; source-code context	Confidentiality, integrity
A-03	Meeting Data	Transcripts/notes and business decisions	Confidentiality, integrity
A-04	Requirements / Documents	Requirements and API documentation	Confidentiality, integrity
A-05	Employee Information	Employee entities and related project context	Confidentiality, integrity, least privilege
A-06	AI Conversations	Manager questions and generated answers	Confidentiality, integrity, auditability
A-07	Knowledge Graph	Entities/relationships connecting employee, task, project, commit, bug, meeting	Integrity, confidentiality, tenant isolation
A-08	Vector Database	Document embeddings and semantic retrieval index	Confidentiality, integrity, tenant isolation
A-09	LLM/RAG Context	Retrieved company context supplied to the LLM	Confidentiality, integrity
A-10	Credentials & Secrets	API keys, passwords, tokens, credentials	Confidentiality, immediate misuse prevention
A-11	Identity / Authorization Data	Users, roles, organizations, permissions	Confidentiality, integrity
A-12	Security/Vulnerability Records	Findings, reports, retest/closure status	Confidentiality, integrity, auditability
A-13	Audit / Monitoring Data	Logs, security events, health signals	Integrity, confidentiality, availability
A-14	Prediction Outputs	Sprint delay, burnout, bug recurrence predictions	Integrity, confidentiality; avoid unauthorized inference
A-15	Integration Credentials / Webhooks	GitHub and future integration secrets/tokens	Confidentiality, integrity

3. Attack Surface Mapping

The attack surface is derived from the documented end-to-end flow and security responsibilities. The project explicitly calls out API/webhook integrations, AuthN/AuthZ, IDOR/Broken Access Control, injection, rate-limit abuse, SSRF, file upload, mobile storage/token handling, prompt injection, unauthorized retrieval, data poisoning, privilege escalation, dependencies and secret exposure, plus final testing with Burp Suite, OWASP ZAP, Postman, Nmap, MobSF and JADX. □filecite□turn1file0□L36-L47□ □filecite□turn1file0□L250-L268□

Surface ID	Attack Surface	Potential Security Entry / Abuse Point	Priority for initial modeling
AS-01	Flutter App	Login/session, API calls, chat, graph, local storage, offline/sync	High
AS-02	Public/API Endpoints	Authentication, authorization, input validation, business operations	Critical
AS-03	GitHub Integration	Webhooks/APIs, repository metadata/content, integration credentials	Critical
AS-04	Document/Meeting Ingestion	Uploaded/imported text, transcripts, requirements, API docs	Critical
AS-05	Event Bus	Published events, background jobs, message contents	High
AS-06	RAG Retrieval	Vector search + Graph retrieval and access filtering	Critical
AS-07	LLM Prompt/Context	Direct and indirect prompt injection; context manipulation	Critical
AS-08	Knowledge Graph	Entity/relationship writes and queries	High
AS-09	Vector DB	Embeddings, metadata, semantic retrieval	Critical
AS-10	Multi-Tenant Data Layer	Tenant identifiers, query scoping, object access	Critical
AS-11	SignalR / Real-Time	Connection/authentication, notification data	High
AS-12	Secrets / Config	Environment variables, Key Vault, integration credentials	Critical
AS-13	Dependencies	NuGet, npm/Flutter, Python packages and known CVEs	High
AS-14	Mobile Binary	APK analysis, local storage, token handling, reverse engineering	High
AS-15	Monitoring / Logs	Sensitive data in logs, log integrity, security-event visibility	Medium/High

4. Trust Boundaries & Data Flow

The following initial trust boundaries are proposed from the documented architecture. Exact deployment/network boundaries must be confirmed with the Backend/DevOps team before treating them as final architecture facts.

TB	Boundary	What crosses it	Primary concern
TB-1	User Device ↔ Backend	Credentials, tokens, API requests, chat questions	Authentication, authorization, TLS, input validation
TB-2	Backend ↔ External Sources	GitHub APIs/webhooks; future integrations	Credential protection, webhook authenticity, SSRF/injection
TB-3	Backend ↔ Event Bus	Events and job payloads	Integrity, authorization, replay/duplication
TB-4	Backend ↔ Data Layer	Tenant-scoped queries and writes	Tenant isolation, IDOR, injection
TB-5	Backend ↔ AI Services	Prompts, retrieved context, documents, graph/vector queries	Data leakage, prompt injection, authorization propagation
TB-6	AI Retrieval ↔ Company Data	Graph/vector retrieval results	Unauthorized retrieval, cross-tenant leakage
TB-7	AI Services ↔ LLM	System/developer context, retrieved data, user query	Prompt injection, indirect injection, sensitive-data leakage
TB-8	Mobile Local Storage	Tokens, cached/offline data	Device compromise, insecure storage

Initial DFD (logical)

External User → Flutter App → Security Controls/AuthN/AuthZ → ASP.NET Backend → Business Logic / APIs → Event Bus → Background Processing → AI Services → Knowledge Graph + Vector DB → RAG → LLM → Backend → Flutter. External data sources (GitHub, Meeting Notes, Requirements/Documents) enter through APIs/webhooks or ingestion paths. SignalR provides real-time delivery to the Flutter app. This follows the project's documented end-to-end flow. filecite turn1file0 L71-L105

5. Initial Threat Model

Threats below are an initial security register for the first threat-modeling draft. They are mapped to the project's explicitly stated security scope and testing targets. Severity is a modeling priority, not a claim that the vulnerability currently exists.

ID	Threat / Scenario	Asset(s)	Affected surface	Security objective	Initial mitigation / control
T-01	Broken authentication / session abuse	A-11, A-01	API, Flutter	AuthN	Strong authentication, secure token handling, session controls, logging
T-02	IDOR / Broken Access Control	A-01,A-05,A-06	API, Graph, Vector	AuthZ	Object-level authorization, tenant-aware access checks, negative tests
T-03	Cross-tenant data leakage	A-01,A-07,A-08	Backend/Data/AI	Isolation	Tenant context enforced server-side on every access path; isolation tests
T-04	Injection into API inputs	A-01,A-07	API	Integrity	Strict validation, parameterization, output encoding, security testing
T-05	Rate-limit abuse / resource exhaustion	A-01,A-13	API/AI	Availability	Rate limiting, quotas, monitoring
T-06	Malicious/forged webhook	A-02,A-01	GitHub integration	Integrity	Webhook authenticity validation, replay protection, least privilege
T-07	SSRF through URL/file/document processing	A-01,A-13	Backend ingestion	Confidentiality/Integrity	Allowlist outbound destinations, safe parsing, network egress controls
T-08	Malicious file/document upload	A-03,A-04	Ingestion	Integrity/Availability	Type/size limits, content validation, safe processing, storage isolation
T-09	Direct prompt injection	A-06,A-09	AI Chat	Integrity/Confidentiality	Prompt-injection detection, instruction separation, authorization outside LLM
T-10	Indirect prompt injection from company content	A-02,A-03,A-04,A-06	RAG ingestion/retrieval	Confidentiality/Integrity	Treat retrieved content as untrusted data; filtering; provenance; output checks
T-11	Unauthorized retrieval	A-01,A-08,A-09	RAG/Graph/Vector	Confidentiality	Permission-aware retrieval; tenant and role filters before context assembly
T-12	Data poisoning	A-07,A-08,A-09	Graph/Vector/RAG	Integrity	Source validation, provenance, ingestion controls, anomaly review
T-13	Sensitive data / secret leakage to LLM	A-10,A-09	DLP/AI	Confidentiality	DLP masking before AI; secret scanning; deny unsafe fields
T-14	Prompt-based privilege escalation	A-11,A-06	LLM/AuthZ	AuthZ	Never let LLM decide authorization; backend policy enforcement
T-15	Knowledge Graph tampering	A-07	Graph	Integrity	Authenticated writes, validation, provenance, authorization
T-16	Vector DB cross-tenant retrieval	A-08	Vector DB	Confidentiality	Tenant metadata filters + server-side enforcement + adversarial tests
T-17	Token/local storage compromise	A-11,A-06	Mobile	Confidentiality	Secure local storage, short-lived credentials, secure logout
T-18	Reverse engineering / APK extraction	A-15,A-10	Mobile	Confidentiality	No embedded secrets, secure storage, minimize sensitive client data
T-19	Dependency with known CVE / supply-chain compromise	A-01,A-10	All builds	Integrity	Dependency inventory, CVE scanning, pinning/updates, review
T-20	Secrets committed to Git history/config	A-10,A-15	Git/CI/CD	Confidentiality	Secret scanning, rotation, Key Vault/env vars, history review
T-21	Sensitive information in logs	A-01,A-06,A-10	Monitoring	Confidentiality	Log redaction, structured logging policy, access control
T-22	Event replay / duplicate processing	A-01,A-07,A-08	Event Bus	Integrity	Idempotency keys, event validation, replay controls

ID	Threat / Scenario	Asset(s)	Affected surface	Security objective	Initial mitigation / control
T-23	Business workflow privilege escalation	A-05,A-06	Backend workflows	Integrity/AuthZ	Server-side workflow authorization and state-transition testing
T-24	AI abuse / excessive inference	A-06,A-14	AI	Confidentiality/Integrity	Rate limits, policy checks, role-based data exposure, monitoring

6. Attack Paths to Model & Test

Path	Attack chain	What must be verified
AP-01	Low-privileged user → API object ID → another tenant's object	Server-side tenant and object authorization cannot be bypassed by changing identifiers.
AP-02	Malicious GitHub content → ingestion → RAG retrieval → LLM context → sensitive answer	Retrieved content is treated as untrusted and cannot override security instructions or expose unauthorized data.
AP-03	Compromised integration credential → GitHub API/webhook → backend → graph/vector	Credential scope, webhook validation, ingestion authorization and provenance limit impact.
AP-04	User prompt → LLM → request for restricted employee/secret data	Authorization is enforced outside the LLM and restricted data never enters unauthorized context.
AP-05	Malicious document/file → parser/ingestion → backend	File type/size/content validation and safe processing prevent parser/SSRF/resource abuse.
AP-06	Mobile compromise → local cache/token → backend APIs	Sensitive data and credentials are protected on the client and server rejects unauthorized use.
AP-07	Dependency compromise/CVE → build/runtime → secrets/data	Dependencies are inventoried/scanned and secrets are not exposed to compromised packages unnecessarily.
AP-08	Event injection/replay → background job → graph/vector update	Events are authenticated/validated, idempotent and cannot cause unauthorized state changes.

STRIDE-style coverage checklist

Category	Project-specific examples to inspect
Spoofing	Login, session/token, webhook identity, SignalR connection identity
Tampering	GitHub events, documents, graph/vector records, business workflows, event messages
Repudiation	Audit logs for authentication, data access, administrative/security actions, ingestion and AI requests
Information Disclosure	Cross-tenant retrieval, employee information, secrets, logs, AI responses, mobile storage
Denial of Service	API rate abuse, AI expensive queries, event flooding, oversized files, background jobs
Elevation of Privilege	IDOR/BAC, role manipulation, workflow abuse, LLM-mediated authorization bypass

7. Security Requirements Derived from the Threat Model

REQ	Requirement
SR-01	Every protected backend resource must enforce server-side authentication and authorization; client-side checks are insufficient.
SR-02	Every tenant-scoped operation must derive/validate tenant context server-side and prevent cross-tenant access.
SR-03	Authorization decisions must not be delegated to the LLM or prompt instructions.
SR-04	RAG retrieval must apply authorization and tenant filtering before content is placed into LLM context.
SR-05	External-source content must be treated as untrusted data, including commit messages, meeting notes and documents.
SR-06	DLP must mask detected API keys/passwords before sensitive data reaches AI processing, consistent with the documented security layer.
SR-07	Integration webhooks must be authenticated/validated and protected against replay/duplicate processing.
SR-08	API endpoints must have input validation and rate limiting, with security tests for injection and abuse.
SR-09	File/document ingestion must constrain size/type/content and prevent unsafe outbound requests.
SR-10	Mobile storage must not expose long-lived sensitive credentials or unnecessary company data.
SR-11	Dependencies across NuGet, npm/Flutter and Python must be inventoried and checked for known CVEs.
SR-12	Secrets must be kept out of source code/Git history and managed through environment variables/Key Vault as planned.
SR-13	Security-relevant events must be logged without leaking secrets or sensitive company content.
SR-14	AI security testing must include prompt injection, indirect prompt injection, unauthorized retrieval, data poisoning and AI abuse scenarios.
SR-15	The vulnerability lifecycle must follow Discover → Validate → Classify → Prioritize → Report → Fix → Retest → Close.
SR-16	Final end-to-end penetration testing must cover web/API, mobile and AI attack paths using the project's listed tools.

8. Security Testing Matrix

Domain	Tests	Documented tools
API Security	Authentication testing; authorization testing; IDOR/Broken Access Control; injection; rate-limit abuse	Burp Suite, OWASP ZAP, Postman
Web/App Security	Broken Access Control; Injection; Security Misconfiguration; SSRF; file upload	Burp Suite, OWASP ZAP
Mobile Security	Local storage; token handling; APK analysis; reverse engineering review	MobSF, JADX, ADB
AI Security	Prompt Injection; Indirect Prompt Injection; unauthorized retrieval; data poisoning; AI abuse	Application-specific test harness + API tooling; Burp/ZAP where applicable
Business Logic	Workflow bypass; privilege escalation; unauthorized state transitions	Burp Suite, Postman + manual test cases
Dependency/Supply Chain	Inventory packages and identify known CVEs	Dependency scanners / package managers; exact scanner to be selected
Secrets Review	Search source code and Git history for API keys/credentials	Secret scanning + Git history review
Vulnerability Management	Discover → Validate → Classify → Prioritize → Report → Fix → Retest → Close	Vulnerability Register
Final E2E	End-to-end attack paths across API, mobile, AI and integrations	Burp Suite, OWASP ZAP, Postman, Nmap, MobSF, JADX

The project specification explicitly lists these security areas and tools, including the final E2E set. □filecite□turn1file0□L250-L268□

9. Professional Threat Model Diagram Specification

For the Word/PDF deliverable, use one main Data Flow Diagram plus a separate attack-surface/asset table. The diagram should visually distinguish external actors, application components, data stores, external systems and trust boundaries. Do not place every individual threat on the architecture diagram; use numbered threat IDs (T-01…T-24) as callouts and keep their explanations in the threat register.

Diagram nodes

External User; Flutter App; Security Middleware; ASP.NET Backend; Authentication/Authorization; API Layer; Business Logic; Event Bus; Background Processing; Multi-Tenant Data Layer; GitHub; Meeting Notes; Requirements/Documents; NLP; Knowledge Graph; Vector DB; RAG; LLM; ML Models; SignalR; Monitoring/Vulnerability Management.

Trust boundaries

TB-1: Device/Client ↔ Backend. TB-2: Backend ↔ External Integrations. TB-3: Backend ↔ Event Bus. TB-4: Backend ↔ Tenant Data. TB-5: Backend ↔ AI Services. TB-6: AI Retrieval ↔ Company Data. TB-7: AI ↔ LLM. TB-8: Mobile Local Storage.

Key attack arrows to label

T-02/T-03: IDOR and cross-tenant access; T-06: forged/replayed webhook; T-09/T-10: direct/indirect prompt injection; T-11/T-16: unauthorized/cross-tenant retrieval; T-13/T-20: secret/data leakage; T-17/T-18: mobile token/storage compromise; T-19: dependency/supply-chain risk; T-22: event replay; T-23: workflow privilege escalation.

10. Assumptions & Verification Points

These are intentionally not presented as implemented facts because the supplied project document does not fully specify them.

ID	Point to confirm with team
V-01	Exact authentication mechanism and token format/lifetime/refresh strategy.
V-02	Exact database technology and schema/tenant-key strategy for the Multi-Tenant Data Layer.
V-03	Whether RabbitMQ or Kafka will be used in the MVP.
V-04	Whether FAISS, Qdrant or Chroma will be used in the final implementation.
V-05	Whether NetworkX or Neo4j will be used in the final implementation.
V-06	Exact LLM provider/model and whether prompts/data leave the deployment boundary.
V-07	Exact file types and upload/import mechanisms for Requirements/Documents/Meeting data.
V-08	Exact GitHub integration mechanism, webhook validation approach and credential scopes.
V-09	Whether the Flutter target is mobile only, web, or both for the security test plan.
V-10	Exact deployment topology, network segmentation, reverse proxy/API gateway and TLS termination.
V-11	Exact logging/audit retention and access model.
V-12	Exact dependency/CVE scanner and CI/CD security gates.

Why this section matters: a threat model should be updated when architecture decisions become concrete. The project document itself distinguishes core MVP scope from roadmap/bonus items, so the security model should follow those implementation decisions rather than inventing components. □filecite□turn1file0□L210-L237□ □filecite□turn1file0□L269-L280□

11. Security Deliverables Checklist for Martina's Track

Deliverable	Status for initial draft	Next action
Critical Assets Inventory	Drafted	Validate with Backend/AI/Flutter owners
Attack Surface Mapping	Drafted	Map actual endpoints/integrations once available
Threat Model Diagram	Draft specification prepared	Render final architecture diagram from confirmed implementation
Threat Register	Initial 24 threats	Review likelihood/impact with team; assign owners
Security Testing Framework	Architecture prepared	Create test cases and acceptance criteria
API Security	In scope	Test AuthN/AuthZ, IDOR/BAC, injection, rate limiting
Web/App Security	In scope	Test OWASP categories listed by project
Mobile Security	In scope	MobSF/JADX/ADB analysis
AI Security	In scope	Prompt injection, indirect injection, unauthorized retrieval, poisoning, abuse
Business Logic Security	In scope	Workflow bypass and privilege escalation
Dependency & Supply Chain	In scope	Inventory + CVE review
Secrets Exposure Review	In scope	Source + Git history + configuration
Vulnerability Management	Defined lifecycle	Maintain official vulnerability register
Final E2E Pentest	Final phase	Burp/ZAP/Postman/Nmap/MobSF/JADX + report

This checklist follows the Security Track scope in the supplied project specification. □filecite□turn1file0□L244-L268□

12. Executive Security Summary

The highest-value security boundary is the path from company-owned data into AI retrieval and generation: external/company content enters through integrations and ingestion, is transformed into graph/vector representations, retrieved by RAG and supplied to the LLM, then returned to an authorized user. The model therefore has to preserve tenant and role authorization across every transformation, not only at the API edge. The project specification explicitly identifies data leakage, prompt injection, unauthorized retrieval, data poisoning and privilege escalation as security scenarios. □filecite□turn1file0□L165-L172□ □filecite□turn1file0□L254-L258□

The second major security boundary is multi-tenancy: the same platform serves multiple software companies, and the specification calls for strict isolation of each company's data. □filecite□turn1file0□L136-L140□

The third is the integration/event-driven layer: GitHub and future integrations can feed asynchronous processing, so webhook authenticity, event integrity, idempotency, credential scope and downstream authorization should be validated before production use.

Document status: Initial Threat Model Draft — ready for team review. It should be versioned after each confirmed architecture change and after security test results are available.